MCE5919: Gaussian Process for Machine Learning and Signal Processing

Lecture 5

Wenqiang PU

Shenzhen Research Institute of Big Data & School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

March 2, 4, 9, 11, 16, 2026

Lecture Goals

- View GP classification as:

$$\text{GP prior on latent function } f(\mathbf{x}) + \text{non-Gaussian likelihood}$$

- Follow a consistent prediction pipeline: latent posterior $\rightarrow$ class probability.
- Implement Laplace approximation:

$$\text{mode } \hat{\mathbf{f}} \rightarrow \text{Gaussian posterior } q(\mathbf{f}) \rightarrow \text{prediction}$$

Notation (Binary Classification)

- Inputs: $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^n$, $\mathbf{x}_i \in \mathbb{R}^d$.
- Labels: $y_i \in \{+1, -1\}$.
- Latent values: $f_i \triangleq f(\mathbf{x}_i)$, vector $\mathbf{f} = [f_1, \ldots, f_n]^\top$.
- Response function $\sigma(\cdot) : \mathbb{R} \to [0, 1]$ (e.g., logistic sigmoid $\sigma(z) = (1 + e^{-z})^{-1}$ or probit $\sigma(z) = \Phi(z)$).

Overview

1. Scope and Background

2. Gaussian Process Classification

3. Laplace Approximation

Scope

- GP regression predicts/outputs real continuous-valued variables
- In this lecture, we will focus on classification problems
  - assign an input pattern $\mathbf{x}$ to one of C classes
- Practical examples: handwritten digit recognition
- We will focus on probabilistic GP classification, where test predictions take the form of class probabilities in contrast to a single guess at the class label.
- Introduce the fundamental ideas with simple binary classification, which can be extended to multi-class straightforwardly

Classification Problems

- By Bayes' Theorem, a joint probability $p(y, \mathbf{x})$ can be decomposed either as

$$p(y)p(\mathbf{x}|y) \text{ or } p(\mathbf{x})p(y|\mathbf{x})$$

- This gives rise to two different approaches to classification problems

Generative approach versus Discriminative approach

Classification Problems

Generative Approach:

- Compute the posterior probability for each class using

$$p(y|\mathbf{x}) = \frac{p(y)p(\mathbf{x}|y)}{\sum_{c=1}^{C} p(\mathcal{C}_c)p(\mathbf{x}|\mathcal{C}_c)}$$

where

- $\mathcal{C}_c,\ c = 1, \ldots, C$: classes
- y : class label $(y = \mathcal{C}_1, \ldots, \mathcal{C}_C)$
- $\mathbf{x}$: samples (features)

Discriminative Approach:

- Directly modeling $p(y|\mathbf{x})$ (without modeling the distribution $p(\mathbf{x}|\mathcal{C}_c)$)

Comparison

Generative Approach — model the joint $p(\mathbf{x}, y)$

- Learn class-conditional density $p(\mathbf{x}|\mathcal{C}_c)$ and class prior $p(\mathcal{C}_c)$, then apply Bayes' rule.
- Example: Gaussian Naive Bayes — assume $p(\mathbf{x}|\mathcal{C}_c) = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}_c, \Sigma_c)$.
- *Pros:* Handles missing inputs; can generate new samples; uses class structure.
- *Con:* Density estimation in high dimensions is hard and error-prone.

Discriminative Approach — model $p(y|\mathbf{x})$ directly

- Example: Logistic regression — $p(y=+1|\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{x}^T\mathbf{w})$.
- *Pro:* Simpler; focuses only on the decision boundary we care about.
- *Con:* Cannot easily handle missing inputs or unlabeled data.

Which to choose?

If prior structure of $p(\mathbf{x}|\mathcal{C}_c)$ is known and useful $\rightarrow$ generative.
Otherwise $\rightarrow$ discriminative.

Binary Discriminative Case

Turning the output of a regression model into a class probability using a response function, mapping from $(-\infty, \infty) \mapsto [0, 1]$.

Example (linear logistic regression)

Combining the linear regression model with the logistic response function.

$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{x}^T\mathbf{w}), \text{ where } \sigma(z) = \frac{1}{1 + \exp(-z)}$$

Binary Discriminative Case

Q & A:

- Q1: What is the response function for multi-class classification?
  $\rightarrow$ Softmax function (a generalization of the logistic function to multiple classes)

$$p(\mathcal{C}_c|\mathbf{x}) = \mathrm{softmax}(\mathbf{x}^T\mathbf{W})_c = \frac{\exp(\mathbf{x}^T\mathbf{w}_c)}{\sum_{j=1}^{C}\exp(\mathbf{x}^T\mathbf{w}_j)},\ c = 1, \ldots, C$$

- Q2: What is the probability prior to the response function?
  $\rightarrow$ Bernoulli distribution (binary) and multinomial distribution (multi-class)

$$p(\mathcal{C}_1|\mathbf{x}) = \sigma(\mathbf{x}^T\mathbf{w}) \quad \text{or} \quad p(\mathcal{C}_c|\mathbf{x}) = \mathrm{softmax}(\mathbf{x}^T\mathbf{W})_c$$

Binary Discriminative Case

Another common choice of response function is the cumulative density function of a standard normal distribution:

$$\Phi(z) = \int_{-\infty}^{z} \mathcal{N}(x|0,1)dx.$$

Remark: This approach is known as probit regression.

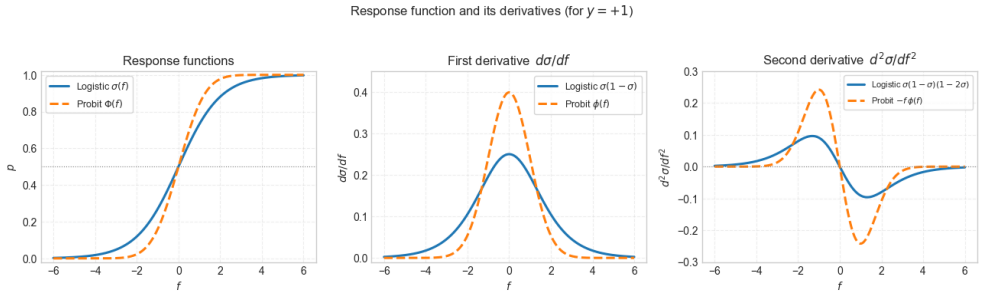


Figure: Response Functions: Logistic vs. Probit .

Linear Models for Classification

For binary classification, with $y = +1$ or $y = -1$, the likelihood is

$$p(y = +1|\mathbf{x}, \mathbf{w}) = \sigma(\mathbf{x}^T\mathbf{w}),$$

- $\mathbf{w}$ is the weight vector
- $\sigma(z)$ is the mapping function
- we have $p(y = -1|\mathbf{x}, \mathbf{w}) = 1 - p(y = +1|\mathbf{x}, \mathbf{w})$.
- linear logistic regression: $\sigma(z) = (1 + \exp(-z))^{-1}$
- linear probit regression: $\sigma(z) = \Phi(z)$
- $\sigma(z) = 1 - \sigma(-z)$, more concisely, for $f_i \triangleq f(\mathbf{x}_i) = \mathbf{x}_i^T\mathbf{w}$,

$$p(y_i|\mathbf{x}_i, \mathbf{w}) = \sigma(y_i f_i)$$

Linear Models for Classification

- Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i) | i = 1, \ldots, n\}$, with prior $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \Sigma_p)$
- The un-normalized log posterior is

$$\log p(\mathbf{w}|X, \mathbf{y}) = -\frac{1}{2}\mathbf{w}^T \Sigma_p^{-1} \mathbf{w} + \sum_{i=1}^{n} \log \sigma(y_i f_i) + const.$$

- For classification, the posterior does not have a simple analytic form.
- For some response functions, both the blue and red parts are concave functions, thus it is easy to find the unique global maximum.

Example — Binary Linear Logistic Regression

Setup

Prior: $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, I)$ in $\mathbb{R}^2$. Likelihood: $p(y_i \mid \mathbf{x}_i, \mathbf{w}) = \sigma(y_i \mathbf{x}_i^\top \mathbf{w})$ (logistic). A small 2-D dataset ($n = 20$) with classes ± 1.

Take-away

The non-Gaussian likelihood makes the posterior non-Gaussian. We *already* need approximation for this simple linear model — the GP extension inherits the same challenge.

Example — Binary Linear Logistic Regression

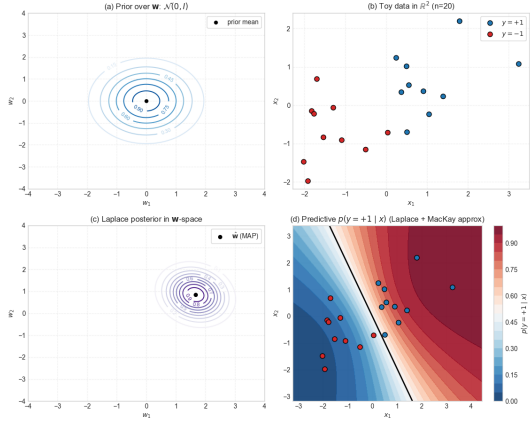


Figure: (a) Prior contours: isotropic Gaussian centred at the origin. (b) Data: two-class scatter plot in $\mathbb{R}^2$. (c) Posterior contours: the prior "squeezed" toward the direction that best separates the classes. (d) Predictive contours: the decision boundary with calibrated class probabilities.

From Linear Classifiers to GP Classification

Linear model (parametric)

Score function is constrained to be linear:

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{w}.$$

Posterior inference is over $\mathbf{w}$.

GP model (nonparametric)

Replace the parametric form by a GP prior over functions:

$$f(\cdot) \sim \mathcal{GP}(0, k), \qquad p(y_i \mid f_i) = \sigma(y_i f_i).$$

Posterior inference is over the training latents $\mathbf{f} = [f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)]^\top$.

Overview

1. Scope and Background

2. Gaussian Process Classification

3. Laplace Approximation

Gaussian Process Classification

Example (Binary classification)

A prior $\pi(\mathbf{x})$ is obtained by "squashing" a GP prior over the latent function $f(\mathbf{x})$

$$\pi(\mathbf{x}) \triangleq p(y = +1|\mathbf{x}) = \sigma(f(\mathbf{x}))$$

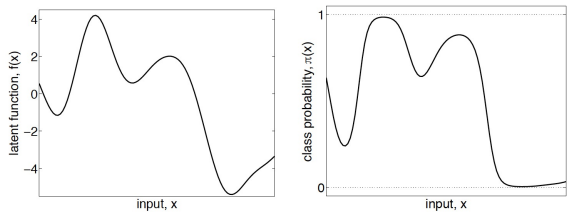


Figure: (a) shows a sample latent function $f(x)$ drawn from a Gaussian process. (b) shows the result of squashing this sample function through the logistic sigmoid function $\sigma(z) = (1 + \exp(-z))^{-1}$ to obtain the class probability $\pi(x) = \sigma(f(x))$.

Generative Model: Regression vs. Classification

GP regression (Gaussian likelihood)

- Latent function prior: $f(\cdot) \sim \mathcal{GP}(0, k)$.
- Observed targets are noisy latent values:

$$y_i = f(\mathbf{x}_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_n^2).$$

- Equivalent likelihood: $p(y_i \mid f_i) = \mathcal{N}(y_i \mid f_i, \sigma_n^2)$ (Gaussian).

GP classification (Bernoulli likelihood + link)

- Latent function prior: $f(\cdot) \sim \mathcal{GP}(0, k)$.
- Observed labels are generated from the latent values via a response function:

$$p(y_i \mid f_i) = \sigma(y_i f_i), \quad y_i \in \{+1, -1\}.$$

- Likelihood is non-Gaussian $\Rightarrow$ posterior is non-Gaussian.

Gaussian Process Classification

What is observed vs. latent?

- Observed inputs: $X = \{\mathbf{x}_i\}_{i=1}^n$.
- Observed labels: $\mathbf{y} = [y_1, \ldots, y_n]^\top$, $y_i \in \{+1, -1\}$.
- Latent (unobserved) function values: $\mathbf{f} = [f(\mathbf{x}_1), \ldots, f(\mathbf{x}_n)]^\top$.
- Test input is $\mathbf{x}_*$, test latent is f_*, and the prediction is a probability

$$p(y_* = +1 \mid \mathbf{x}_*, X, \mathbf{y}) \longleftrightarrow \int \sigma(f_*) p(f_* | X, \mathbf{y}, \mathbf{x}_*) df_*$$

Remark: Law of Total Probability $P(A) = \int P(A|B)P(B)dB$

Gaussian Process Classification

Suppose that we have selected a proper kernel function and hyper-parameters

Inference is naturally divided into two steps:

- **Step 1:** compute the marginal distribution of the latent variable at the test point:

$$p(f_*|X, \mathbf{y}, \mathbf{x}_*) = \int p(f_*|X, \mathbf{x}_*, \mathbf{f})\, p(\mathbf{f}|X, \mathbf{y})\, d\mathbf{f},$$

  where $p(\mathbf{f}|X, \mathbf{y}) = p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|X)/p(\mathbf{y}|X)$ is the posterior over training latents.
- **Step 2:** use the distribution of f_* to produce a probabilistic prediction:

$$\bar{\pi}_* \triangleq p(y_* = +1|X, \mathbf{y}, \mathbf{x}_*) = \int \sigma(f_*)\, p(f_*|X, \mathbf{y}, \mathbf{x}_*)\, df_*.$$

  Unlike the regression case, the integral is in general non-Gaussian, which makes it analytically intractable.

Step 1: Derivation of the Integral

i: Marginalization

- Introduce latent variables $\mathbf{f}$ and integrate out:

$$p(f_*|X, \mathbf{y}, \mathbf{x}_*) = \int p(f_*, \mathbf{f}|X, \mathbf{y}, \mathbf{x}_*)\, d\mathbf{f}$$

ii: Chain Rule (Factorization)

- Decompose the joint distribution:

$$p(f_*, \mathbf{f}|X, \mathbf{y}, \mathbf{x}_*) = p(f_*|\mathbf{f}, X, \mathbf{y}, \mathbf{x}_*) \cdot p(\mathbf{f}|X, \mathbf{y}, \mathbf{x}_*)$$

iii: Conditional Independence

- Given $\mathbf{f}$, f_* is **independent** of $\mathbf{y}$: $p(f_*|\mathbf{f}, X, \mathbf{y}, \mathbf{x}_*) = p(f_*|\mathbf{f}, X, \mathbf{x}_*)$
- $\mathbf{f}$'s posterior is **independent** of $\mathbf{x}_*$: $p(\mathbf{f}|X, \mathbf{y}, \mathbf{x}_*) = p(\mathbf{f}|X, \mathbf{y})$

iv: Final Result

$$\boxed{p(f_*|X, \mathbf{y}, \mathbf{x}_*) = \int p(f_*|X, \mathbf{x}_*, \mathbf{f})\, p(\mathbf{f}|X, \mathbf{y})\, d\mathbf{f}}$$

Step 1: Intuition of the Integral

Why integrate over $\mathbf{f}$?

We do *not* know the exact training latent values $\mathbf{f}$. We must average the test prediction over *all plausible* $\mathbf{f}$, weighted by how probable each $\mathbf{f}$ is given the observed labels $\mathbf{y}$.

Each factor in the integral has a clear role:

- $p(f_*|X, \mathbf{x}_*, \mathbf{f})$ — **GP prior predictive**: given latents $\mathbf{f}$, predicts f_* via the kernel.
- $p(\mathbf{f}|X, \mathbf{y})$ — **latent posterior**: updates our belief about $\mathbf{f}$ after seeing labels $\mathbf{y}$.
- $\int(\cdot)\, d\mathbf{f}$ — **marginalise**: removes the unobserved nuisance variable $\mathbf{f}$.

Step 1: Intuition of the Integral

Analogy: Imagine predicting tomorrow's weather by averaging forecasts from many different weather models.

- **Historical weather conditions (e.g. pressure)** $\longleftrightarrow$ Training inputs X
- **Observed weather outcomes (e.g. rain/sun)** $\longleftrightarrow$ Observed labels $\mathbf{y}$
- **A specific weather model parameters** $\longleftrightarrow$ A specific latent vector $\mathbf{f}$ (r.v.)
- **Probability of rain given the forecast** $\longleftrightarrow$ $\sigma(f_*)$
- **Forecast from that model** $\longleftrightarrow$ Prediction given latents $p(f_*|X, \mathbf{x}_*, \mathbf{f})$
- **Model's reliability (based on past data)** $\longleftrightarrow$ Posterior weight $p(\mathbf{f}|X, \mathbf{y})$
- **Final consensus forecast** $\longleftrightarrow$ The integral result

$$\int \sigma(f_*) \int p(f_*|X, \mathbf{x}_*, \mathbf{f})\, p(\mathbf{f}|X, \mathbf{y})\, d\mathbf{f}\; df_*$$

Step 1: Posterior Factorization

Bayes rule

$$p(\mathbf{f} \mid X, \mathbf{y}) \propto p(\mathbf{y} \mid \mathbf{f}, X)\, p(\mathbf{f} \mid X).$$

Conditional independence (the key modeling assumption)

Given the latent values $\mathbf{f}$, labels do not depend on inputs X:

$$p(\mathbf{y} \mid \mathbf{f}, X) = p(\mathbf{y} \mid \mathbf{f}) = \prod_{i=1}^{n} p(y_i \mid f_i).$$

This yields $p(\mathbf{f} \mid X, \mathbf{y}) \propto p(\mathbf{f} \mid X)\, p(\mathbf{y} \mid \mathbf{f})$.

Step 2: The Integral Non-Gaussian

In GP regression

Both the prior $p(\mathbf{f}|X)$ and the likelihood $p(\mathbf{y}|\mathbf{f})$ are Gaussian
$\Rightarrow$ posterior $p(\mathbf{f}|X, \mathbf{y})$ is Gaussian $\Rightarrow$ all integrals are analytic.

In GP classification

The likelihood $p(y_i|f_i) = \sigma(y_i f_i)$ is a sigmoid/probit function — **not Gaussian**.
$\Rightarrow$ posterior $p(\mathbf{f}|X, \mathbf{y}) \propto p(\mathbf{f}|X) \prod_i \sigma(y_i f_i)$ is non-Gaussian.
$\Rightarrow$ the integral $\bar{\pi}_* = \int \sigma(f_*) p(f_*|\cdots)\, df_*$ has **no closed form**.

Bottom line: The non-Gaussianity comes entirely from the shape of the response function applied to the latent variable. We must resort to approximations.

Overview

1. Scope and Background

2. Gaussian Process Classification

3. Laplace Approximation

Laplace Approximation: The Big Picture

General Approaches

Adopt either analytic approximations of integral or solutions based on Monte Carlo sampling.

Idea of Laplace Approximation

Approximate the intractable posterior $p(\mathbf{f}|X,\mathbf{y})$ by a Gaussian $q(\mathbf{f}|X,\mathbf{y})$ around its mode.

$$p(\mathbf{f}|X,\mathbf{y}) \approx q(\mathbf{f}|X,\mathbf{y})$$

1. Find the mode $\hat{\mathbf{f}} = \arg\max_{\mathbf{f}} \log p(\mathbf{y}|\mathbf{f}) + \log p(\mathbf{f}|X)$.
2. Form Gaussian posterior $q(\mathbf{f}) = \mathcal{N}(\hat{\mathbf{f}}, (K^{-1} + W)^{-1})$.
3. Predict latent $q(f_*|\mathbf{x}_*, X, \mathbf{y})$.
4. Predict label $\bar{\pi}_* = \int \sigma(f_*)\, q(f_*)\, df_*$ (averaged) or $\hat{\pi}_* = \sigma(\mathbb{E}_q[f_*])$ (MAP).

Laplace Approximation — Intuition

Why expand at the *mode* $\hat{\mathbf{f}}$?

The mode is the single most probable point. A Gaussian centered there captures the **shape** of the posterior near its peak — the region that contributes most to any integral.

Why a *second-order* Taylor expansion?

- **0th order** ($\Psi(\hat{\mathbf{f}})$): just a constant — no shape information.
- **1st order** ($\nabla\Psi = \mathbf{0}$ at the mode): vanishes by definition; contributes nothing.
- **2nd order** ($-\frac{1}{2}(\mathbf{f}-\hat{\mathbf{f}})^T A(\mathbf{f}-\hat{\mathbf{f}})$): gives a Gaussian shape, where A (the **Hessian**) encodes curvature (i.e., how sharply the posterior falls off).

Laplace Approximation

- Laplace's method uses a Gaussian approximation $q(\mathbf{f}|X,\mathbf{y})$ to the posterior $p(\mathbf{f}|X,\mathbf{y})$.
- Do a second-order Taylor expansion of $\log p(\mathbf{f}|X,\mathbf{y})$ around its mode to get:

$$p(\mathbf{f} \mid X,\mathbf{y}) \approx q(\mathbf{f}|X,\mathbf{y}) = \mathcal{N}(\mathbf{f}|\hat{\mathbf{f}}, A^{-1}) \propto \exp\Big(-\tfrac{1}{2}(\mathbf{f}-\hat{\mathbf{f}})^T A(\mathbf{f}-\hat{\mathbf{f}})\Big),$$

where $\hat{\mathbf{f}} = \arg\max_{\mathbf{f}}\ p(\mathbf{f}|X,\mathbf{y})$ and $A = -\nabla\nabla \log p(\mathbf{f}|X,\mathbf{y})|_{\mathbf{f}=\hat{\mathbf{f}}}$.

Connecting A and $K^{-1}+W$

Since $\log p(\mathbf{f}|X,\mathbf{y}) = \Psi(\mathbf{f}) + \text{const}$, we have $A = -\nabla\nabla\Psi\big|_{\hat{\mathbf{f}}} = K^{-1} + W$.

Log-Posterior $\Psi(\mathbf{f})$ and Its Derivatives

By Bayes' rule $p(\mathbf{f}|X,\mathbf{y}) = p(\mathbf{y}|\mathbf{f})p(\mathbf{f}|X)/p(\mathbf{y}|X)$, the log posterior $\log p(\mathbf{f}|X,\mathbf{y})$ is

$$\Psi(\mathbf{f}) \triangleq \log p(\mathbf{y}|\mathbf{f}) + \log p(\mathbf{f}|X)$$

Differentiating w.r.t. $\mathbf{f}$, yields

$$\nabla\Psi(\mathbf{f}) = \nabla \log p(\mathbf{y}|\mathbf{f}) - K^{-1}\mathbf{f},$$
$$\nabla\nabla\Psi(\mathbf{f}) = \nabla\nabla \log p(\mathbf{y}|\mathbf{f}) - K^{-1} = -W - K^{-1},$$

where $W \triangleq -\nabla\nabla \log p(\mathbf{y}|\mathbf{f})$ is a diagonal matrix and $K \triangleq k(X,X)$ is the $n \times n$ kernel matrix

Note

If the likelihood $p(\mathbf{y}|\mathbf{f})$ is log concave, then $\Psi(\mathbf{f})$ is concave and has a unique global maximum.

Likelihood Derivatives — Explicit Formulas

For binary labels $y_i \in \{+1, -1\}$, the likelihood is $p(y_i \mid f_i) = \sigma(y_i f_i)$. Define

$$t_i \triangleq y_i f_i, \quad \pi_i \triangleq \sigma(t_i) = \sigma(y_i f_i).$$

Logistic $\sigma(f) = \frac{1}{1+e^{-f}}$

$$\begin{aligned} \pi_i &= \sigma(y_i f_i) \\ \nabla_{f_i} \log p(y_i \mid f_i) &= y_i(1 - \pi_i) \\ W_{ii} \triangleq -\nabla^2_{f_i} \log p(y_i \mid f_i) &= \pi_i(1 - \pi_i) \end{aligned}$$

Probit $\Phi(f) = \int_{-\infty}^{f} \mathcal{N}(t|0,1)dt$

$$\begin{aligned} \pi_i &= \Phi(y_i f_i) \\ \nabla_{f_i} \log p(y_i|f_i) &= y_i \frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)} \\ W_{ii} &= \left(\frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)}\right)^2 + y_i f_i \frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)} \end{aligned}$$

Derivation: Logistic Likelihood

Goal: Compute $\nabla_{f_i} \log \sigma(y_i f_i)$ and $\nabla^2_{f_i} \log \sigma(y_i f_i)$ for $\sigma(z) = (1 + e^{-z})^{-1}$.

Key Property: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

1. First Derivative (Gradient):

$$\log p(y_i|f_i) = \log \sigma(y_i f_i)$$

$$\frac{\partial}{\partial f_i} \log \sigma(y_i f_i) = \frac{1}{\sigma(y_i f_i)} \cdot \sigma'(y_i f_i) \cdot y_i \quad \text{(Chain rule)}$$

$$= \frac{1}{\pi_i} \cdot [\pi_i(1 - \pi_i)] \cdot y_i = \boxed{y_i(1 - \pi_i)}$$

2. Second Derivative (Hessian Element $-W_{ii}$):

$$\frac{\partial^2}{\partial f_i^2} \log p(y_i|f_i) = \frac{\partial}{\partial f_i}[y_i - y_i \sigma(y_i f_i)]$$

$$= 0 - y_i \cdot \sigma'(y_i f_i) \cdot y_i = -y_i^2 \pi_i(1 - \pi_i)$$

$$W_{ii} = -(-\pi_i(1 - \pi_i)) = \boxed{\pi_i(1 - \pi_i)} \quad (\text{since } y_i^2 = 1)$$

Derivation: Probit Likelihood

Goal: Compute derivatives for $\Phi(z) = \int_{-\infty}^{z} \mathcal{N}(x|0,1)dx$.

Key Property: $\Phi'(z) = \mathcal{N}(z)$ and $\mathcal{N}'(z) = \mathcal{N}(z) \cdot \frac{d}{dz}\left(-\frac{z^2}{2}\right) = -z\mathcal{N}(z)$

1. First Derivative:

$$\frac{\partial}{\partial f_i} \log \Phi(y_i f_i) = \frac{1}{\Phi(y_i f_i)} \Phi'(y_i f_i) y_i = \boxed{y_i \frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)}}$$

2. Second Derivative: Let $z = y_i f_i$ Then:

$$\begin{aligned} \frac{\partial}{\partial f_i}\left(y_i \frac{\mathcal{N}(z)}{\Phi(z)}\right) &= y_i^2 \frac{\mathcal{N}'(z)\Phi(z) - \mathcal{N}(z)\Phi'(z)}{(\Phi(z))^2} \quad (y_i^2 = 1, \text{ quotient rule}) \\ &= \frac{-z\mathcal{N}(z)\Phi(z) - (\mathcal{N}(z))^2}{\Phi(z)^2} = -\left(\frac{\mathcal{N}(z)^2}{\Phi(z)^2} + z\frac{\mathcal{N}(z)}{\Phi(z)}\right) \end{aligned}$$

Thus, $W_{ii} = -\frac{\partial^2}{\partial f_i^2}(\dots) = \boxed{\left[\frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)}\right]^2 + y_i f_i \frac{\mathcal{N}(y_i f_i)}{\Phi(y_i f_i)}}$

Likelihood Derivatives — Explicit Formulas

Why do we need W?

W is the **curvature** (precision) of the log-likelihood. It enters the Newton update and determines the covariance of the Laplace Gaussian: $\Sigma = (K^{-1} + W)^{-1}$. A large W_{ii} means observation i strongly constrains f_i.

Mode Finding via Newton's Method

- At the maximum of $\Psi(\mathbf{f})$, we have

$$\nabla\Psi(\mathbf{f}) = \mathbf{0} \Longrightarrow \hat{\mathbf{f}} = K(\nabla \log p(\mathbf{y}|\hat{\mathbf{f}})),$$

  as a self-consistent equation for $\hat{\mathbf{f}}$, but $\log p(\mathbf{y}|\hat{\mathbf{f}})$ is a non-linear function of $\hat{\mathbf{f}}$.
- To find the maximum of $\Psi(\mathbf{f})$, we often resort to Newton's method

$$\begin{aligned}
\mathbf{f}^{new} &= \mathbf{f} - (\nabla\nabla\Psi)^{-1}\nabla\Psi \\
&= \mathbf{f} + (K^{-1} + W)^{-1}(\nabla \log p(\mathbf{y}|\mathbf{f}) - K^{-1}\mathbf{f}) \\
&= (K^{-1} + W)^{-1}(W\mathbf{f} + \nabla \log p(\mathbf{y}|\mathbf{f})).
\end{aligned}$$

Derivation sketch (line by line)

Line 1: standard Newton update.
Line 2: substitute $\nabla\nabla\Psi = -(K^{-1} + W)$ and $\nabla\Psi = \nabla \log p(\mathbf{y}|\mathbf{f}) - K^{-1}\mathbf{f}$.
Line 3: multiply out $\mathbf{f} + (K^{-1} + W)^{-1}(-K^{-1}\mathbf{f})$ and collect $W\mathbf{f} = (K^{-1} + W)\mathbf{f} - K^{-1}\mathbf{f}$.

Newton's Method vs. Gradient Ascent

Goal: find the mode $\hat{\mathbf{f}} = \arg\max_{\mathbf{f}} \Psi(\mathbf{f})$

For GP classification, $\Psi(\mathbf{f}) = \log p(\mathbf{y} \mid \mathbf{f}) + \log p(\mathbf{f} \mid X)$ is typically concave (with log-concave likelihood), so there is a unique maximum.

Gradient ascent

$$\mathbf{f}_{k+1} = \mathbf{f}_k + \eta \nabla\Psi(\mathbf{f}_k)$$

Step size η must be tuned; can be slow if curvature varies.

Newton's method

$$\mathbf{f}_{k+1} = \mathbf{f}_k - \left(\nabla\nabla\Psi(\mathbf{f}_k)\right)^{-1} \nabla\Psi(\mathbf{f}_k)$$

Uses curvature (W) so it typically converges in fewer iterations, but each step is more expensive.

In our case

$\nabla\nabla\Psi(\mathbf{f}) = -(K^{-1} + W)$, so each Newton step solves a linear system with an SPD matrix.

Newton's Method

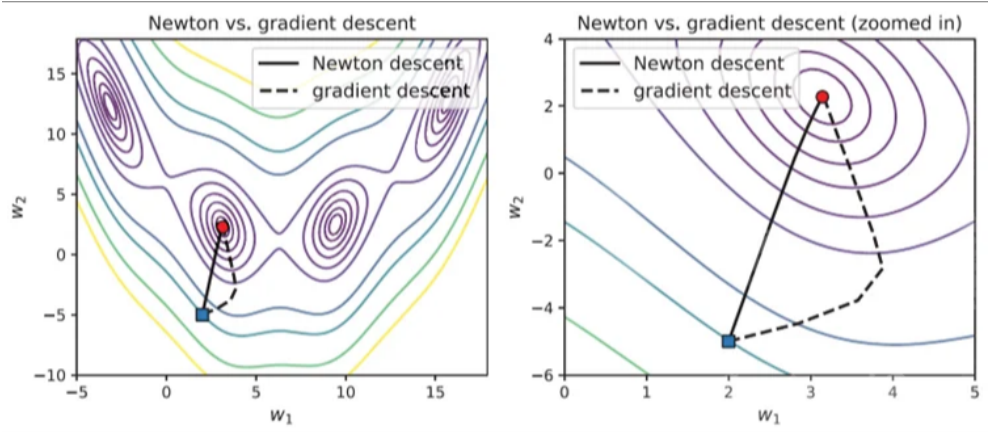
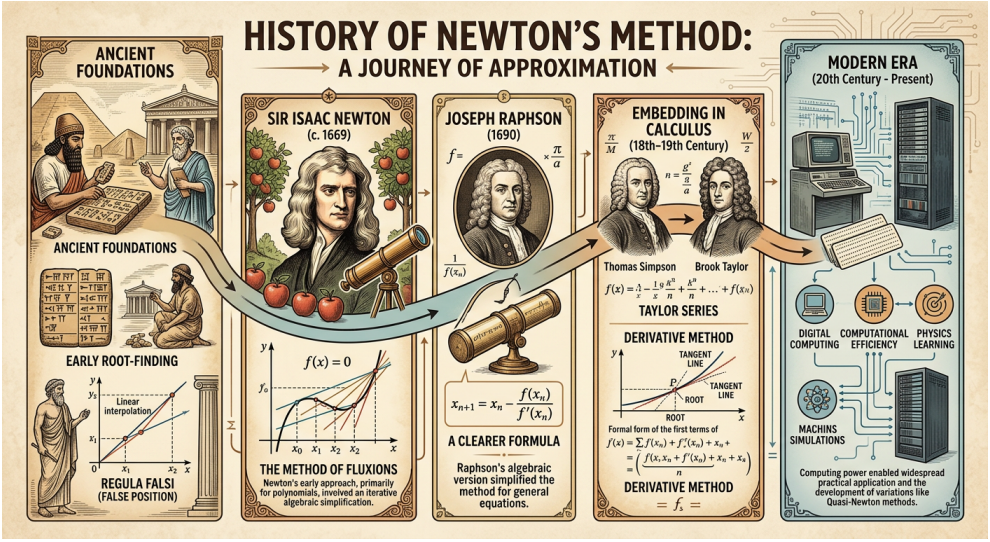


Figure: Convergence Curve of Newton and Gradien Methods.

Newton's Method - Brief History



The Laplace-Approximate Posterior

Having found the mode $\hat{\mathbf{f}}$, we can now specify the Laplace approximation to the posterior as a Gaussian

$$q(\mathbf{f}|X, \mathbf{y}) = \mathcal{N}(\hat{\mathbf{f}}, (K^{-1} + W)^{-1}).$$

Note

The Laplace method is essentially uncontrolled, the Hessian (evaluated at $\hat{\mathbf{f}}$) may give a poor approximation to the true shape of the posterior.

From Posterior to Prediction

What we have so far

$$q(\mathbf{f} \mid X, \mathbf{y}) = \mathcal{N}(\hat{\mathbf{f}}, (K^{-1} + W)^{-1})$$

What we still need

1. The predictive distribution $q(f_* \mid \mathbf{x}_*, X, \mathbf{y})$ for a new test point $\mathbf{x}_*$.
2. A class probability $\bar{\pi}_* = \int \sigma(f_*)\, q(f_*)\, df_*$ or $\hat{\pi}_* = \sigma(\mathbb{E}_q[f_*])$.

Prediction: Total Expectation & Total Variance

Key identity (conditioning on training latents $\mathbf{f}$)

Under the GP prior, the test latent depends on data only through $\mathbf{f}$:

$$p(f_* \mid \mathbf{x}_*, X, \mathbf{y}) = \int p(f_* \mid \mathbf{x}_*, X, \mathbf{f})\, p(\mathbf{f} \mid X, \mathbf{y})\, d\mathbf{f}.$$

Law of total expectation / variance

$$\mathbb{E}[f_* \mid \mathbf{x}_*, X, \mathbf{y}] = \mathbb{E}_{p(\mathbf{f}\mid X,\mathbf{y})}\big[\mathbb{E}[f_* \mid \mathbf{x}_*, X, \mathbf{f}]\big],$$

$$\mathrm{Var}[f_* \mid \mathbf{x}_*, X, \mathbf{y}] = \mathbb{E}\big[\mathrm{Var}[f_* \mid \mathbf{x}_*, X, \mathbf{f}]\big] + \mathrm{Var}\big(\mathbb{E}[f_* \mid \mathbf{x}_*, X, \mathbf{f}]\big).$$

Total Variance

Starting from the definition $\mathrm{Var}(Y) = \mathbb{E}[Y^2] - (\mathbb{E}[Y])^2$, and applying the law of total expectation $\mathbb{E}[Y] = \mathbb{E}[\mathbb{E}[Y|X]]$:

1. Expand $\mathbb{E}[Y^2]$ using iterated expectation:

$$\mathbb{E}[Y^2] = \mathbb{E}\big[\mathbb{E}[Y^2 \mid X]\big]$$

2. Decompose the inner conditional second moment:

$$\mathbb{E}[Y^2 \mid X] = \mathrm{Var}(Y \mid X) + \big(\mathbb{E}[Y \mid X]\big)^2$$

3. Substitute back:

$$\mathbb{E}[Y^2] = \mathbb{E}\big[\mathrm{Var}(Y \mid X)\big] + \mathbb{E}\Big[\big(\mathbb{E}[Y \mid X]\big)^2\Big]$$

4. Subtract $(\mathbb{E}[Y])^2 = \big(\mathbb{E}[\mathbb{E}[Y|X]]\big)^2$:

$$\mathrm{Var}(Y) = \underbrace{\mathbb{E}\big[\mathrm{Var}(Y \mid X)\big]}_{\text{within}} + \underbrace{\mathbb{E}\Big[\big(\mathbb{E}[Y \mid X]\big)^2\Big] - \Big(\mathbb{E}\big[\mathbb{E}[Y \mid X]\big]\Big)^2}_{\mathrm{Var}(\mathbb{E}[Y|X]),\ \text{between}}$$

Prediction — Posterior Mean

The exact posterior mean for a test point $\mathbf{x}_*$ is

$$\begin{aligned}\mathbb{E}_p\left[f_* \mid \mathbf{x}_*, \mathcal{D}\right] &= \mathbb{E}_{p(\mathbf{f}|X,\mathbf{y})}\Big[\mathbb{E}\big[f_* \mid \mathbf{x}_*, X, \mathbf{f}\big]\Big] \\ &= \mathbb{E}_{p(\mathbf{f}|X,\mathbf{y})}\big[\mathbf{k}(\mathbf{x}_*)^T K^{-1}\mathbf{f}\big] \\ &= \mathbf{k}(\mathbf{x}_*)^T K^{-1}\mathbb{E}\left[\mathbf{f} \mid X, \mathbf{y}\right],\end{aligned}$$

We obtain the Laplace approximated posterior mean as

$$\mathbb{E}_q\left[f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}\right] = \mathbf{k}(\mathbf{x}_*)^T K^{-1}\hat{\mathbf{f}}.$$

Prediction — Posterior Variance

The exact posterior variance for a test point $\mathbf{x}_*$ is

Law of total variance (conditioning on $\mathbf{f}$)

$$\mathrm{Var}_p\big[f_* \mid \mathbf{x}_*, X, \mathbf{y}\big] = \mathbb{E}\big[\mathrm{Var}(f_* \mid \mathbf{x}_*, X, \mathbf{f})\big] + \mathrm{Var}\big(\mathbb{E}[f_* \mid \mathbf{x}_*, X, \mathbf{f}]\big).$$

For a GP prior

$$\mathbb{E}[f_* \mid \mathbf{x}_*, X, \mathbf{f}] = \mathbf{k}(\mathbf{x}_*)^T K^{-1}\mathbf{f}, \qquad \mathrm{Var}(f_* \mid \mathbf{x}_*, X, \mathbf{f}) = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^T K^{-1}\mathbf{k}(\mathbf{x}_*).$$

Prediction — Laplace Approximated Variance

We obtain the Laplace approximated posterior variance as

$$\begin{aligned}\mathrm{Var}_q\left[f_* \mid \mathbf{x}_*, \mathbf{X}, \mathbf{y}\right] &= \underbrace{k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^T K^{-1} \mathbf{k}(\mathbf{x}_*)}_{\text{GP conditional variance (given } \mathbf{f})} \\ &\quad + \underbrace{\mathbf{k}(\mathbf{x}_*)^T K^{-1} \left(K^{-1} + W\right)^{-1} K^{-1} \mathbf{k}(\mathbf{x}_*)}_{\text{extra uncertainty from not knowing } \mathbf{f}} \\ &= k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^T \left(K + W^{-1}\right)^{-1} \mathbf{k}(\mathbf{x}_*).\end{aligned}$$

Remark — intermediate algebra

- The first term is $\mathbb{E}[\mathrm{Var}(f_* \mid \mathbf{x}_*, X, \mathbf{f})] = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^T K^{-1} \mathbf{k}(\mathbf{x}_*)$ (cons. w.r.t. $\mathbf{f}$).
- The second term is $\mathrm{Var}_q(\mathbf{k}(\mathbf{x}_*)^T K^{-1} \mathbf{f}) = \mathbf{k}(\mathbf{x}_*)^T K^{-1} \underbrace{(K^{-1} + W)^{-1}}_{\text{Laplace covariance}} K^{-1} \mathbf{k}(\mathbf{x}_*)$.
- Combining and applying the matrix inversion lemma yields the compact last line.

Prediction — Averaged vs. MAP

Given the mean and variance of f_*, we can make predictions by computing

1. The averaged prediction

$$\bar{\pi}_* \simeq \mathbb{E}_q[\pi_* | X, \mathbf{y}, \mathbf{x}_*] = \int \sigma(f_*) q(f_* | X, \mathbf{y}, \mathbf{x}_*) df_*,$$

   where $q(f_* | X, \mathbf{y}, \mathbf{x}_*)$ is Gaussian with mean $\mathbb{E}_q[f_*]$ and variance $\mathbb{V}_q[f_*]$ derived above.

2. The MAP prediction

$$\hat{\pi}_* = \sigma(\arg\max_{f_*} q(f_* | X, \mathbf{y}, \mathbf{x}_*)) = \sigma(\mu_*)$$
$$\mu_* \triangleq \mathbb{E}_q[f_* \mid \mathbf{x}_*, X, \mathbf{y}] = \mathbf{k}(\mathbf{x}_*)^T K^{-1} \hat{\mathbf{f}}$$

Prediction — Averaged vs. MAP

Remark: how to compute the integral

If we choose a **probit** response $\Phi(\cdot)$, then

$$\int \Phi(f_*)\,\mathcal{N}(f_* \mid \mu_*, \sigma_*^2)\,df_* = \Phi\left(\frac{\mu_*}{\sqrt{1+\sigma_*^2}}\right).$$

For the **logistic** sigmoid $\sigma(\cdot)$, the integral has no closed form; a common approximation is

$$\int \sigma(f_*)\,\mathcal{N}(f_* \mid \mu_*, \sigma_*^2)\,df_* \approx \sigma\left(\frac{\mu_*}{\sqrt{1+\pi\sigma_*^2/8}}\right),$$

or we can use Gauss–Hermite quadrature numerically.

Averaged vs. MAP Prediction — Intuition

MAP prediction $\hat{\pi}_* = \sigma(\mathbb{E}_q[f_*])$

Plug the mean latent value directly into the response function.
Fast and simple, but *ignores* uncertainty in f_*.
If $q(f_*)$ is narrow (high confidence), MAP $\approx$ averaged.

Averaged prediction $\bar{\pi}_* = \int \sigma(f_*)\, q(f_*)\, df_*$

Weight every possible latent value by how probable it is, then average.
This is the *correct* Bayesian prediction.
Because $\sigma(\cdot)$ is **nonlinear**, $\bar{\pi}_* \neq \sigma(\mathbb{E}_q[f_*])$ in general.
The two coincide only when $\mathbb{V}_q[f_*] \to 0$.

Key take-away

When uncertainty is large, averaged prediction is more calibrated; MAP can be over-confident.

Practical Implementation

- The mode-finding procedure uses the Newton iteration, involving the matrix

$$\left(K^{-1} + W\right)^{-1}.$$

- Using the matrix inversion lemma, we get

$$\left(K^{-1} + W\right)^{-1} = K - KW^{\frac{1}{2}}B^{-1}W^{\frac{1}{2}}K,$$

where

$$B = I + W^{\frac{1}{2}}KW^{\frac{1}{2}},$$

computation of which costs only $\mathcal{O}(n^2)$, since W is diagonal.

- Matrix B is positive definite, thus it is numerically stable to conduct the Cholesky decomposition $LL^\top = B$ and compute $B^{-1}\mathbf{z}$.

Practical Implementation — Intuition

Why reformulate $(K^{-1} + W)^{-1}$ via B?

Directly inverting an $n \times n$ matrix costs $\mathcal{O}(n^3)$. Forming $B = I + W^{1/2} K W^{1/2}$ requires only $\mathcal{O}(n^2)$ extra work (since W is diagonal), and its Cholesky factor L can be reused everywhere.

Practical Implementation

input: K (covariance matrix), $\mathbf{y}$ (± 1 targets), $p(\mathbf{y}|\mathbf{f})$ (likelihood function)
$\mathbf{f} := \mathbf{0}$ initialization
repeat Newton iteration
  $W := -\nabla\nabla \log p(\mathbf{y}|\mathbf{f})$ eval. W e.g. using eq. (3.15) or (3.16)
  $L := \text{cholesky}(I + W^{\frac{1}{2}} K W^{\frac{1}{2}})$ $B = I + W^{\frac{1}{2}} K W^{\frac{1}{2}}$
  $\mathbf{b} := W\mathbf{f} + \nabla \log p(\mathbf{y}|\mathbf{f})$
  $\mathbf{a} := \mathbf{b} - W^{\frac{1}{2}} L^{\top} \backslash (L \backslash (W^{\frac{1}{2}} K \mathbf{b}))$ } eq. (3.18) using eq. (3.27)
  $\mathbf{f} := K\mathbf{a}$
until convergence objective: $-\frac{1}{2}\mathbf{a}^{\top}\mathbf{f} + \log p(\mathbf{y}|\mathbf{f})$
$\log q(\mathbf{y}|X, \theta) := -\frac{1}{2}\mathbf{a}^{\top}\mathbf{f} + \log p(\mathbf{y}|\mathbf{f}) - \sum_i \log L_{ii}$ eq. (3.32)
return: $\hat{\mathbf{f}} := \mathbf{f}$ (post. mode), $\log q(\mathbf{y}|X, \theta)$ (approx. log marg. likelihood)

Figure: Implementation for finding the Laplace approximation, model-finding for binary Laplace GPC.

Summary

1. **GP Classification** extends GP regression by replacing the Gaussian likelihood with a Bernoulli likelihood + response function $\sigma(\cdot)$.
2. The posterior $p(\mathbf{f} \mid X, \mathbf{y})$ is non-Gaussian $\Rightarrow$ intractable integrals.
3. **Laplace Approximation**:
   - Find the mode $\hat{\mathbf{f}}$ via Newton's method.
   - Approximate $p(\mathbf{f} \mid X, \mathbf{y}) \approx q(\mathbf{f}) = \mathcal{N}(\hat{\mathbf{f}}, (K^{-1} + W)^{-1})$.
4. **Prediction** via the law of total expectation/variance:
   - Mean: $\mathbb{E}_q[f_*] = \mathbf{k}(\mathbf{x}_*)^T K^{-1} \hat{\mathbf{f}}$.
   - Variance: $\mathrm{Var}_q[f_*] = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^T (K + W^{-1})^{-1} \mathbf{k}(\mathbf{x}_*)$.
   - Class probability: averaged $\bar{\pi}_* = \int \sigma(f_*) q(f_*) df_*$ or MAP $\hat{\pi}_* = \sigma(\mathbb{E}_q[f_*])$.
5. **Marginal likelihood** approximation enables kernel hyperparameter optimisation.
6. Stable implementation via Cholesky of $B = I + W^{1/2} K W^{1/2}$.

The End

Appendix

Laplace Approximation for Multi-class GP Classification

From Binary to Multi-class: The Big Picture

Binary classification (reviewed)

- **One** latent function $f(\mathbf{x})$; response: logistic $\sigma(f)$; labels $y \in \{+1, -1\}$.
- Hessian W is **diagonal** — each training point contributes independently.

Multi-class extension (C classes)

- **One latent function per class**: $f^c(\mathbf{x})$, $c = 1, \ldots, C$; response: softmax.
- Labels $y_i \in \{1, \ldots, C\}$; one-hot encoding: $y_{ic} \triangleq \mathbf{1}[y_i = c]$.

Key new challenge

Softmax **couples** all C latent values at each input — the Hessian W is no longer diagonal, making the Newton update more involved.

Notation for Multi-class GP Classification

- Inputs: $X = \{\mathbf{x}_i\}_{i=1}^n$, $\mathbf{x}_i \in \mathbb{R}^d$.
- Labels: $y_i \in \{1, \ldots, C\}$; one-hot: $y_{ic} = \mathbf{1}[y_i = c]$.
- **Latent value** for class c at point i: $f_i^c \triangleq f^c(\mathbf{x}_i)$.
- **Latent vector** for class c (over all training points): $\mathbf{f}^c = [f_1^c, \ldots, f_n^c]^\top \in \mathbb{R}^n$.
- **Latent vector** at point i (over all classes): $\mathbf{f}_i = [f_i^1, \ldots, f_i^C]^\top \in \mathbb{R}^C$.
- **Full latent matrix**: $F = [\mathbf{f}^1, \ldots, \mathbf{f}^C] \in \mathbb{R}^{n \times C}$.
- **Vectorized latents**: $\mathbf{f} = \mathrm{vec}(F) = [(\mathbf{f}^1)^\top, \ldots, (\mathbf{f}^C)^\top]^\top \in \mathbb{R}^{nC}$.
- **Softmax probability**: $\pi_{ic} = \dfrac{\exp(f_i^c)}{\sum_{j=1}^C \exp(f_i^j)}$, vector $\boldsymbol{\pi}_i = [\pi_{i1}, \ldots, \pi_{iC}]^\top \in \mathbb{R}^C$.

GP Prior and Softmax Likelihood for Multi-class

Independent GPs — one per class

$$f^c(\cdot) \sim \mathcal{GP}(0, k), \quad c = 1, \ldots, C$$

Training latents per class: $\mathbf{f}^c \sim \mathcal{N}(\mathbf{0}, K)$, $K = k(X, X) \in \mathbb{R}^{n \times n}$.
Joint prior of $\mathbf{f} = \mathrm{vec}(F)$:

$$p(\mathbf{f} \mid X) = \prod_{c=1}^{C} \mathcal{N}(\mathbf{f}^c \mid \mathbf{0}, K), \qquad K_{\mathbf{f}} = I_C \otimes K.$$

GP Prior and Softmax Likelihood for Multi-class

Softmax likelihood (generalizes binary sigmoid)

$$p(y_i = c \mid \mathbf{f}_i) = \pi_{ic} = \mathrm{softmax}(\mathbf{f}_i)_c = \frac{\exp(f_i^c)}{\sum_{j=1}^{C} \exp(f_i^j)}$$

$$\log p(y_i \mid \mathbf{f}_i) = \sum_{c=1}^{C} y_{ic} f_i^c - \log \sum_{j=1}^{C} \exp(f_i^j)$$

Softmax Derivatives — Gradient and Hessian

First derivative (gradient w.r.t. f_i^c):

$$\frac{\partial \log p(y_i \mid \mathbf{f}_i)}{\partial f_i^c} = y_{ic} - \pi_{ic}$$

Intuition: gradient vanishes when predicted probability matches the one-hot label.

Second derivative (Hessian w.r.t. f_i^c and $f_i^{c'}$):

$$\frac{\partial^2 \log p(y_i \mid \mathbf{f}_i)}{\partial f_i^c \, \partial f_i^{c'}} = -\pi_{ic}\big(\delta_{cc'} - \pi_{ic'}\big)$$

In **matrix form**, the $C \times C$ Hessian at point i is:

$$\nabla_{\mathbf{f}_i} \nabla_{\mathbf{f}_i}^\top \log p(y_i \mid \mathbf{f}_i) = -\underbrace{\big(\operatorname{diag}(\boldsymbol{\pi}_i) - \boldsymbol{\pi}_i \boldsymbol{\pi}_i^\top\big)}_{\triangleq M_i \succeq 0}$$

Derivation: Softmax Gradient and Hessian

Setup: Let $Z_i = \sum_j \exp(f_i^j)$, so $\pi_{ic} = \exp(f_i^c)/Z_i$.

1. Gradient:

$$\frac{\partial}{\partial f_i^c} \log p(y_i \mid \mathbf{f}_i) = \frac{\partial}{\partial f_i^c}\Big(\sum_{c'} y_{ic'} f_i^{c'} - \log Z_i\Big) = y_{ic} - \frac{\exp(f_i^c)}{Z_i} = \boxed{y_{ic} - \pi_{ic}}$$

2. Hessian: Differentiate again, using the quotient rule on $\pi_{ic} = \exp(f_i^c)/Z_i$:

$$\frac{\partial \pi_{ic}}{\partial f_i^{c'}} = \frac{\delta_{cc'} \exp(f_i^c) \cdot Z_i - \exp(f_i^c)\exp(f_i^{c'})}{Z_i^2} = \pi_{ic}(\delta_{cc'} - \pi_{ic'})$$

$$\frac{\partial^2 \log p(y_i \mid \mathbf{f}_i)}{\partial f_i^c \, \partial f_i^{c'}} = -\frac{\partial \pi_{ic}}{\partial f_i^{c'}} = \boxed{-\pi_{ic}(\delta_{cc'} - \pi_{ic'})}$$

Derivation: Softmax Gradient and Hessian

Matrix form (c indexes rows, c' indexes columns):

$$\nabla_{\mathbf{f}_i} \nabla_{\mathbf{f}_i}^\top \log p(y_i \mid \mathbf{f}_i) = -(\mathrm{diag}(\boldsymbol{\pi}_i) - \boldsymbol{\pi}_i \boldsymbol{\pi}_i^\top)$$

Note: $M_i = \mathrm{diag}(\boldsymbol{\pi}_i) - \boldsymbol{\pi}_i \boldsymbol{\pi}_i^\top \succeq 0$ (covariance matrix of a categorical distribution).

Key difference from binary

M_i has off-diagonal entries $-\pi_{ic}\pi_{ic'}$ (the softmax couples all classes).
Compare: binary has a single scalar $w_i = \pi_i(1 - \pi_i)$.

Structure of W — The Full Hessian

Define $W = -\nabla_{\mathbf{f}} \nabla_{\mathbf{f}}^{\top} \log p(\mathbf{y} \mid F) \in \mathbb{R}^{nC \times nC}$ (vectorizing by class).

W is a $C \times C$ block matrix, where block (c, c') is an $n \times n$ **diagonal** matrix:

$$W = \begin{pmatrix} D_{11} & D_{12} & \cdots & D_{1C} \\ D_{21} & D_{22} & \cdots & D_{2C} \\ \vdots & & \ddots & \vdots \\ D_{C1} & D_{C2} & \cdots & D_{CC} \end{pmatrix}, \qquad (D_{cc'})_{ii} = \pi_{ic}(\delta_{cc'} - \pi_{ic'}).$$

Binary case (recap)

$$W = \mathrm{diag}(w_1, \ldots, w_n)$$

$w_i = \pi_i(1 - \pi_i)$; W is $n \times n$ diagonal.

Multi-class case

$$W \in \mathbb{R}^{nC \times nC}, \text{ block structure}$$

Diagonal blocks: $D_{cc} = \mathrm{diag}(\pi_{ic}(1 - \pi_{ic}))$.
Off-diagonal blocks: $D_{cc'} = -\mathrm{diag}(\pi_{ic}\pi_{ic'})$.

Log-Posterior $\Psi(\mathbf{f})$ and Its Derivatives

By Bayes' rule, the log-posterior (up to a constant) is

$$\Psi(\mathbf{f}) \triangleq \log p(\mathbf{y} \mid F) - \frac{1}{2}\mathbf{f}^\top K_{\mathbf{f}}^{-1}\mathbf{f} = \sum_{i=1}^{n} \log p(y_i \mid \mathbf{f}_i) - \frac{1}{2}\sum_{c=1}^{C} (\mathbf{f}^c)^\top K^{-1}\mathbf{f}^c.$$

Differentiating w.r.t. $\mathbf{f}$:

$$\nabla_{\mathbf{f}}\Psi = \nabla_{\mathbf{f}} \log p(\mathbf{y} \mid F) - K_{\mathbf{f}}^{-1}\mathbf{f},$$
$$\nabla_{\mathbf{f}}\nabla_{\mathbf{f}}^\top \Psi = -(W + K_{\mathbf{f}}^{-1}).$$

where the **gradient vector** (organized by class) is

$$\nabla_{\mathbf{f}} \log p(\mathbf{y} \mid F) = \begin{bmatrix} \mathbf{y}_{\cdot 1} - \boldsymbol{\pi}_{\cdot 1} \\ \vdots \\ \mathbf{y}_{\cdot C} - \boldsymbol{\pi}_{\cdot C} \end{bmatrix} \in \mathbb{R}^{nC}, \quad \mathbf{y}_{\cdot c} = [y_{1c}, \ldots, y_{nc}]^\top, \ \boldsymbol{\pi}_{\cdot c} = [\pi_{1c}, \ldots, \pi_{nc}]^\top.$$

$\Psi(\mathbf{f})$ is concave since $W \succeq 0$; Newton's method finds the unique global mode.

Mode Finding via Newton's Method

The Newton update has the **same structural form** as in the binary case:

$$\mathbf{f}^{\text{new}} = (K_{\mathbf{f}}^{-1} + W)^{-1}\big(W\mathbf{f} + \nabla_{\mathbf{f}} \log p(\mathbf{y} \mid F)\big)$$

Derivation (same algebra as binary)

Line 1: standard Newton step $\mathbf{f}^{\text{new}} = \mathbf{f} - (\nabla\nabla\Psi)^{-1}\nabla\Psi$.
Line 2: substitute $\nabla\nabla\Psi = -(K_{\mathbf{f}}^{-1} + W)$ and $\nabla\Psi = \nabla_{\mathbf{f}} \log p(\mathbf{y}|F) - K_{\mathbf{f}}^{-1}\mathbf{f}$.
Line 3: multiply out and collect $W\mathbf{f} = (K_{\mathbf{f}}^{-1} + W)\mathbf{f} - K_{\mathbf{f}}^{-1}\mathbf{f}$.

Computational cost vs. binary

The system $(K_{\mathbf{f}}^{-1} + W)^{-1}$ is now $nC \times nC$ instead of $n \times n$. Each Newton step costs $\mathcal{O}((nC)^3)$ — a factor of C^3 more expensive than binary.

The Laplace-Approximate Posterior

Having found the mode $\hat{\mathbf{f}}$, the Laplace approximation is

$$q(\mathbf{f} \mid X, \mathbf{y}) = \mathcal{N}\Big(\hat{\mathbf{f}},\ (K_{\mathbf{f}}^{-1} + W)^{-1}\Big)$$

where W is evaluated at $\hat{\mathbf{f}}$.

Side-by-side comparison: binary vs. multi-class

	Binary	**Multi-class**
Latent dimension	n	nC
Prior covariance	K	$K_{\mathbf{f}} = I_C \otimes K$
W structure	diagonal $n \times n$	block $nC \times nC$
Posterior covariance	$(K^{-1} + W)^{-1}$	$(K_{\mathbf{f}}^{-1} + W)^{-1}$
Response function	logistic $\sigma(\cdot)$ / probit	softmax$(\cdot)$

Prediction — Latent Posterior at a Test Point

Under the Laplace approximation, for a test input $\mathbf{x}_*$, the predictive distribution of the latent vector $\mathbf{f}_* = [f_*^1, \ldots, f_*^C]^\top$ is Gaussian:

$$q(\mathbf{f}_* \mid \mathbf{x}_*, X, \mathbf{y}) = \mathcal{N}(\boldsymbol{\mu}_*, \Sigma_*)$$

Predictive mean (per class, via law of total expectation)

$$\mu_*^c \triangleq \mathbb{E}_q[f_*^c \mid \mathbf{x}_*, X, \mathbf{y}] = \mathbf{k}(\mathbf{x}_*)^\top K^{-1} \hat{\mathbf{f}}^c, \quad c = 1, \ldots, C.$$

Predictive variance (per class, via law of total variance)

$$\mathbb{V}_q[f_*^c \mid \mathbf{x}_*, X, \mathbf{y}] = k(\mathbf{x}_*, \mathbf{x}_*) - \mathbf{k}(\mathbf{x}_*)^\top K^{-1} \mathbf{k}(\mathbf{x}_*) + \mathbf{k}(\mathbf{x}_*)^\top K^{-1} \Sigma^{cc} K^{-1} \mathbf{k}(\mathbf{x}_*)$$

where Σ^{cc} is the (c, c)-th $n \times n$ block of $(K_{\mathbf{f}}^{-1} + W)^{-1}$. (Cross-class blocks are non-zero due to W coupling.)

Class Probability Prediction — Averaged vs. MAP

The exact class probability for test point $\mathbf{x}_*$ is:

$$p(y_* = c \mid \mathbf{x}_*, X, \mathbf{y}) = \int \mathrm{softmax}(\mathbf{f}_*)_c \, q(\mathbf{f}_* \mid \mathbf{x}_*, X, \mathbf{y}) \, d\mathbf{f}_*$$

Since softmax is **nonlinear**, this integral has no closed form.

MAP prediction (fast)

$$\hat{p}(y_* = c) = \mathrm{softmax}(\boldsymbol{\mu}_*)_c = \frac{\exp(\mu_*^c)}{\sum_{j=1}^{C} \exp(\mu_*^j)}$$

Plug the *posterior mean* $\boldsymbol{\mu}_*$ into softmax.

Averaged prediction (more accurate)

$$\bar{p}(y_* = c) \approx \frac{1}{S} \sum_{s=1}^{S} \mathrm{softmax}\left(\mathbf{f}_*^{(s)}\right)_c$$

Draw $\mathbf{f}_*^{(s)} \sim q(\mathbf{f}_*)$; average over S Monte Carlo samples.

Take-away: same principle as binary

When uncertainty in $\mathbf{f}_*$ is large, **averaged prediction is better calibrated** than MAP. The two agree only when $\mathbb{V}_q \to 0$.

Summary: Multi-class Laplace Approximation

1. **One GP per class**: $f^c(\cdot) \sim \mathcal{GP}(0, k)$; vectorize into $\mathbf{f} \in \mathbb{R}^{nC}$; prior $K_{\mathbf{f}} = I_C \otimes K$.
2. **Softmax likelihood**: $p(y_i = c \mid \mathbf{f}_i) = \mathrm{softmax}(\mathbf{f}_i)_c$; log-likelihood is

$$\log p(y_i \mid \mathbf{f}_i) = \textstyle\sum_c y_{ic} f_i^c - \log \sum_j \exp(f_i^j).$$

3. **Gradient and Hessian**: $\nabla_{f_i^c} = y_{ic} - \pi_{ic}$; Hessian block $M_i = \mathrm{diag}(\boldsymbol{\pi}_i) - \boldsymbol{\pi}_i \boldsymbol{\pi}_i^\top$ (non-diagonal).
4. **Same Laplace recipe — larger scale**:
   - Find mode $\hat{\mathbf{f}}$ via Newton: $\mathbf{f}^{\text{new}} = (K_{\mathbf{f}}^{-1} + W)^{-1}(W\mathbf{f} + \nabla_{\mathbf{f}} \log p(\mathbf{y} \mid F))$.
   - Posterior: $q(\mathbf{f}) = \mathcal{N}(\hat{\mathbf{f}}, (K_{\mathbf{f}}^{-1} + W)^{-1})$.
5. **Prediction**:
   - Mean per class: $\mu_*^c = \mathbf{k}(\mathbf{x}_*)^\top K^{-1} \hat{\mathbf{f}}^c$.
   - Class probability: MAP via $\mathrm{softmax}(\boldsymbol{\mu}_*)$, or averaged via Monte Carlo.
6. **Cost**: $\mathcal{O}((nC)^3)$ vs. $\mathcal{O}(n^3)$ for binary — $C^3\times$ more expensive.